

# Chapter 4

## Exception Handling

# Objectives

- At the end of this chapter students should be able:
  - To get an overview of exceptions and exception handling.
  - To distinguish exception types: **Error** (fatal) vs. **Exception** (nonfatal) and checked vs. unchecked.
  - To declare exceptions in a method header.
  - To throw exceptions in a method.
  - To write a **try-catch - finally** block to handle exceptions.
  - To create chained exceptions.
  - To define custom exception classes.

# Introduction to Exception

- Runtime errors occur while a program is running if the JVM detects an operation that is impossible to carry out.
- For example, if you access an array using an index that is out of bounds, you will get a runtime error with an **ArrayIndexOutOfBoundsException**.
- If you enter a double value when your program expects an integer, you will get a runtime error with an **InputMismatchException**.
- An exception is an object that represents an error or a condition that occurs during a program's execution.

# Introduction to Exception...

- If the exception is not handled, the program will terminate abnormally.
- Exception handling enables programmers to create applications that can resolve exceptions.
- When an exception occurs, the normal execution flow is interrupted and the exception handling code is executed.

# Java Exception Hierarchy

- Exception classes inherit directly or indirectly from class *Exception*, forming an inheritance hierarchy.
- Class *Exception* and its subclasses represent exceptional situations that can occur in a Java program. (e.g division of number by zero)

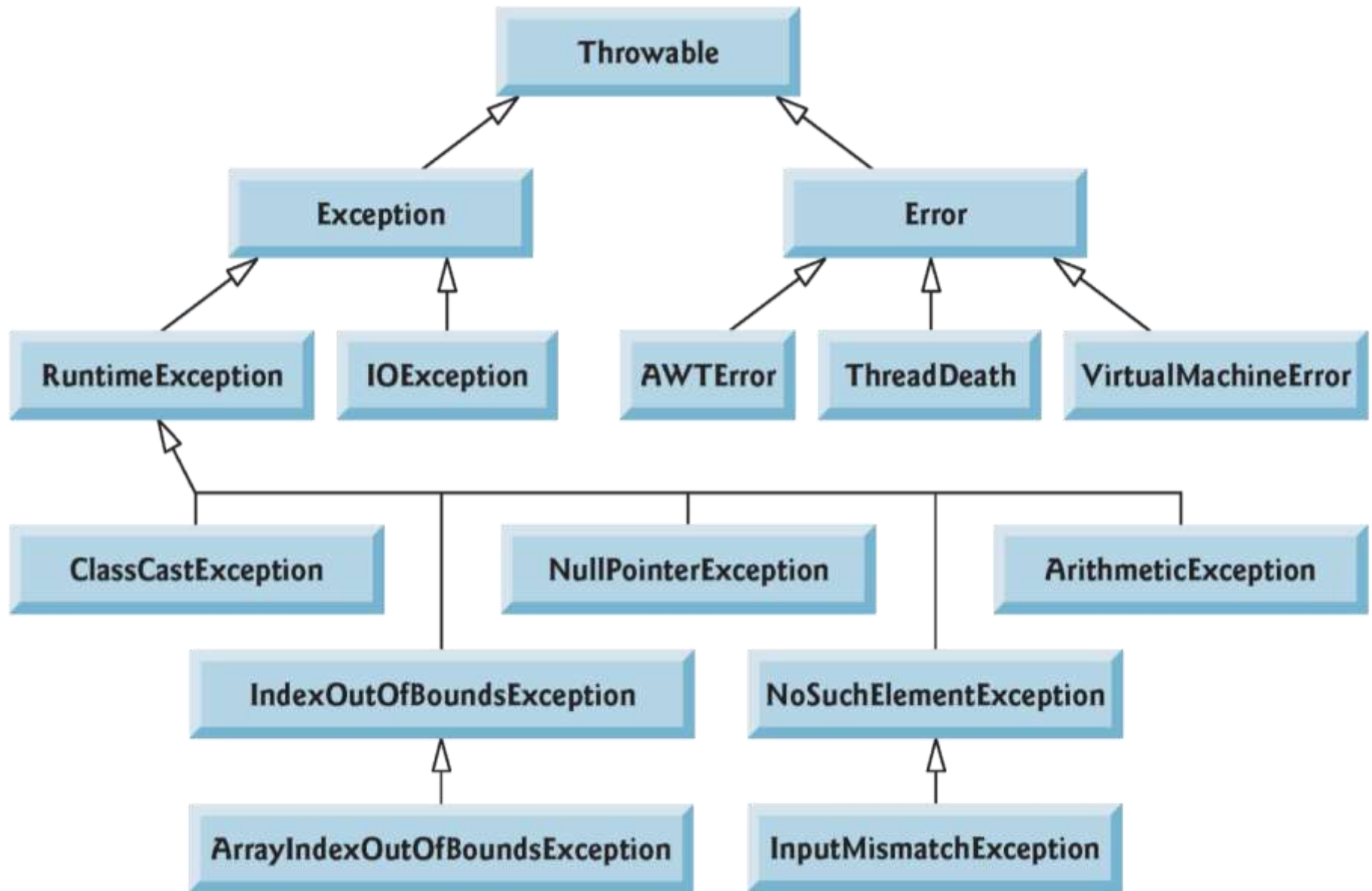
These can be caught and handled by the application.

- Class *Error* and its subclasses represent abnormal situations that happen in the JVM.

*Errors* happen infrequently.

These should not be caught by applications.

Applications usually cannot recover from *Errors*.



# Checked vs. Unchecked Exceptions

- The **Throwable** class is the root of exception classes.
- All Java exception classes inherit directly or indirectly from Throwable.
- You can create your own exception classes by extending Exception or a subclass of Exception.
- The exception classes can be classified into three major types: **system errors or error**, **exceptions**, and **runtime exceptions**.
- RuntimeException, Error and their subclasses are known as ***unchecked exceptions***.

# Checked vs. Unchecked Exceptions

- These are typically caused by defects in your program's code such as bad casting, accessing an out-of-bounds array, and numeric errors.
- Unchecked exceptions typically can be prevented by proper coding.
- All classes that inherit from class **Exception** but not directly or indirectly from class `RuntimeException` are considered to be **checked exceptions**,
- meaning that the compiler forces the programmer to check and deal with the exceptions in a try-catch block or declare it in the method header.

# Exercise (What is the output??)

Output????



```
public class Test {  
    public static void main(String[] args) {  
        int[] list = new int[5];  
        System.out.println(list[5]);  
    }  
}
```

ArrayIndexOutOfBoundsException

```
public class Test {  
    public static void main(String[] args) {  
        String s = "abc";  
        System.out.println(s.charAt(3));  
    }  
}
```

StringIndexOutOfBoundsException

```
public class Test {  
    public static void main(String[] args) {  
        Object o = new Object();  
        String d = (String)o;  
    }  
}
```

ClassCastException

```
public class Test {  
    public static void main(String[] args) {  
        Object o = null;  
        System.out.println(o.toString());  
    }  
}
```

NullPointerException

# Exception-Handling Overview

- Exception handling helps you write robust and fault-tolerant programs that can deal with problems and continue executing or terminate gracefully.
- Exceptions are thrown from a method.
- The caller of the method can catch and handle the exception.
- Only classes that directly or indirectly extend class ***Throwable*** (package ***java.lang***) can be used in exception handling.
- Java's exception-handling model is based on three operations: declaring an exception, throwing an exception, and catching an exception

# Declaring, Throwing and Catching Exceptions

Declare exception

```
method1( ) {
```

```
    try {  
        Invoke method2;  
    }  
    catch (Exception ex) {  
        Process exception;  
    }  
}
```

Catch exception

```
method2 ( )
```

```
    throws Exception {
```

```
    if (an error occurs) {
```

```
        throw new Exception( );
```

```
    }  
}
```

Throw exception

# Declaring Exceptions

- Every method must state the types of **checked exceptions** it might throw.
- Java does not require that you declare unchecked exceptions explicitly in the method
- This is known as *declaring exceptions*.

## ⑩ Example:

```
public void myMethod() throws IOException {
```

```
    ...
```

```
}
```

// comma separated list of exceptions

```
public void myMethod() throws IOException, OtherException {
```

```
    ...
```

```
}
```

# Throwing Exceptions

- When the program detects an error, the program can create an instance of an appropriate exception type and throw it.
- This is known as ***throwing an exception***.

Here is an example:

```
throw new TheException();
```

Or

```
TheException ex = new TheException();  
throw ex;
```

## **When to Throw Exceptions?**

- An exception occurs in a method.
  - If you want the exception to be processed by its caller, you should create an exception object and throw it.
  - If you can handle the exception in the method where it occurs, there is no need to throw it.

# Catching Exceptions

- When an exception is thrown, it can be caught and handled in a try – catch block.
- If no exceptions arise during the execution of the try block, the catch blocks are skipped.
- If one of the statements inside the try block throws an exception, Java skips the remaining statements in the try block and starts the process of finding the code to handle the exception.
- The code that handles the exception is called the **exception handler**; it is found by propagating the exception backward through a chain of method calls.
- The process of finding a handler is called **catching an exception**.

# Try – catch block syntax:

```
try{  
    statements; // Statements that may throw exceptions  
}  
catch(Exception1 exVar1) {  
    handler for exception1;  
}  
catch(Exception2 exVar2) {  
    handler for exception2;  
}  
...  
catch(ExceptionN exVarN) {  
    handler for exceptionN;  
}
```

# Catching Exceptions...

- The compiler does not check the code to determine whether an unchecked exception is caught or declared:
  - If an unchecked exception occurs and is not caught, the program terminates or runs with unexpected results.
  - These situations typically can be prevented by proper coding.

# Catching Exceptions...

- If multiple *catch* blocks match a particular exception type, only the first matching *catch* block executes.
- It's a compilation error to catch the exactly same type in two different *catch* blocks associated with a particular *try* block.
- It's also a compilation error to have the preceding *catch* block to catch an exception type that is a superclass of the exception type declared in any subsequent *catch* block.

# Catching Exceptions...

- Positioning a catch block for the superclass type after all other subclass catch blocks ensures that all subclass exceptions are eventually caught.

```
try {  
    ...  
}  
catch (Exception ex) {  
    ...  
}  
catch (RuntimeException ex) {  
    ...  
}
```

(a) Wrong order

```
try {  
    ...  
}  
catch (RuntimeException ex) {  
    ...  
}  
catch (Exception ex) {  
    ...  
}
```

(b) Correct order

# Catch or Declare Checked Exceptions

Suppose method p2 is defined as follows:

```
void p2( ) {  
    if (a file does not exist) {  
        throw new IOException("File does not exist");  
    }  
    ...  
}
```

- Java forces you to deal with checked exceptions.
- If a method declares a checked exception (*as p2*), you must invoke it in a **try-catch** block or **declare to throw** the exception in the calling method.

# Catch or Declare Checked Exceptions...

➤ For example, suppose that method **p1** invokes method **p2** and p2 may throw a checked exception (e.g., `IOException`), you have to write the code as shown in (a) or (b).

```
void p1() {  
    try {  
        p2();  
    }  
    catch (IOException ex) {  
        ...  
    }  
}
```

(a) Catch exception

```
void p1() throws IOException {  
    p2();  
}
```

(b) Throw exception

## Example: Divide by Zero without Exception Handling

- Java does not allow division by zero in integer arithmetic. JVM throws an *ArithmeticException*.
- Java does allow division by zero with floating point values.

Floating point value displays this as *Infinity*.

If 0.0 is divided by 0.0, the result is *NaN* (not a number), which is represented as a floating point value.

- An *InputMismatchException* occurs when *Scanner* method *nextInt* receives a *String* that does not represent a valid integer.

# Handling *ArithmeticExceptions* and *InputMismatchExceptions*

- Now we can use exception handling to process any *ArithmeticExceptions* and *InputMistmatchExceptions* that arise.
- If the user makes a mistake, the program catches and handles (i.e., deals with) the exception, in this case, allowing the user to try to enter the input again.
- JVM uses the **System.err (standard error stream) object** to output error messages.

# Multi-catch

- If the bodies of several *catch* blocks are identical, you can use the multi-catch feature (introduced in Java SE 7) to catch those exception types in a single *catch* handler and perform the same task.
- The syntax for a multi-catch is:  
`catch (Type1 | Type2 | Type3 e)`
- Each exception type is separated from the next with a vertical bar (|).
- Any number of *Throwable* types can be specified in a multi-catch.

# Uncaught exception

- **Uncaught exception** is the one for which there are no matching *catch* blocks.
- An exception that's not caught in an application causes Java's default exception handler to run.
- This displays the name of the exception, a descriptive message that indicates the problem that occurred and a complete execution stack trace.

In an application with a single thread of execution, the application terminates.

In an application with multiple threads, only the thread that caused the exception terminates.

# Uncaught exception... example

```
Please enter an integer numerator: 100
Please enter an integer denominator: 0
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at DivideByZeroNoExceptionHandling.quotient(
        DivideByZeroNoExceptionHandling.java:10)
    at DivideByZeroNoExceptionHandling.main(
        DivideByZeroNoExceptionHandling.java:22)
```

- The user enters the value 0 as the denominator.
- Several lines of information are displayed in response to this invalid input.
- This information is known as a **stack trace**
- This helps you debug the program.

## Rethrowing Exceptions

- Java allows an exception handler to rethrow the exception if the handler cannot process the exception or simply wants to let its caller be notified of the exception.

- **Re-throw an exception:**

```
try {  
    statements;  
}  
catch(TheException ex) {  
    perform operations before exits;  
    throw ex;  
}
```

- ⑩ The statement `throw ex` rethrows the exception to the caller so that other handlers in the caller get a chance to process the exception `ex`.

# The finally Clause / Block

- The *finally* block is normally used for resource de-allocation.
- Consists of *finally* keyword followed by a block of resource release code enclosed in curly braces.  
resource mean files, database connections and network connections
- It must be placed after the last *catch* block.
- Always executes whether or not an exception is thrown in the corresponding *try* block or any of its corresponding *catch* blocks.
- *finally* block will execute even if a *try* or *catch* block exits by using a *return*, *break* or *continue* statement or simply by reaching its closing right brace.
- *finally* block will *not* execute if the application terminates immediately by calling *System.exit* method.

# The finally Clause / Block...

- A try statement with a finally block:

```
try
{
    statements
    resource-acquisition statements
} // end try
catch ( AKindOfException exception1 )
{
    exception-handling statements
} // end catch
...
catch ( AnotherKindOfException exception2 )
{
    exception-handling statements
} // end catch
finally
{
    statements
    resource-release statements
} // end finally
```

---

# Trace a Program Execution

```
try {
```

```
  statements;
```

```
}
```

```
catch(TheException ex) {
```

```
  handling ex;
```

```
}
```

```
finally {
```

```
  final Statements;
```

```
}
```

```
Next statement;
```

1. Suppose no exceptions in the statements

2. The final block is always executed

3. Next statement in the method is executed

## Trace a Program Execution...

```
try {
```

```
    statement1;
```

```
    statement2;
```

```
    statement3;
```

```
}
```

```
catch(Exception1 ex) {
```

```
    handling ex;
```

```
}
```

```
finally {
```

```
    final Statements;
```

```
}
```

```
Next statement;
```

1. Suppose an exception of type Exception1 is thrown in statement2

2. The exception is handled.

3. The final block is always executed.

4. The next statement in the method is now executed.

# Trace a Program Execution...

```
try {  
    statement1;  
    statement2;  
    statement3;  
}  
catch(Exception1 ex) {  
    handling ex;  
}  
catch(Exception2 ex) {  
    handling ex;  
    throw ex;  
}  
finally {  
    final Statements;  
}  
Next statement;
```

1. statement2 throws an exception of type Exception2.

2. Handling exception

4. Rethrow the exception and control is transferred to the caller

3. Execute the final block

# Exercise (what will be the output??)

```
1 public class FinallyDemo {  
2     public static void main(String[] args) {  
3         try{  
4             throw new ClassCastException();  
5         }  
6     }  
7     catch(ClassCastException ex1){  
8         System.out.println("statement in catch block");  
9         System.exit(0);  
10    }  
11    finally  
12    {  
13    }  
14 }  
15 }  
16 }
```

```
1 public class FinallyDemo {  
2     public static void main(String[] args) {  
3         try{  
4             //throw new ClassCastException();  
5         }  
6     }  
7     catch(ClassCastException ex1){  
8         System.out.println("statement in catch block");  
9         System.exit(0);  
10    }  
11    finally{  
12        System.out.println("statement in finally block");  
13    }  
14 }  
15 }  
16 }
```

```
1 public class FinallyDemo {  
2     public static void main(String[] args) {  
3         try{  
4             throw new ClassCastException();  
5         }  
6     }  
7     catch(ClassCastException ex1){  
8         System.out.println("statement in catch block");  
9         // System.exit(0);  
10    }  
11    finally{  
12        System.out.println("statement in finally block");  
13    }  
14 }  
15 }  
16 }
```

# When to Use Exception Handling?

When should you use the try-catch block in the code?

- To deal with unexpected error conditions or **synchronous errors**, which occur when a statement executes, such as division by zero, out-of-range array indices, invalid method parameters, file I/O problems, network connectivity issues...
- ❖ Do not use the try-catch block to deal with simple, expected situations or problems associated with **asynchronous regular processing events**:
  - disk I/O completions.
  - network message arrivals.
  - mouse clicks and keystrokes.
- ❖ Avoid using exception handling as an alternate form of flow of control.

# When to Use . . .

- Do not use a try-catch block to deal with simple, expected situations.
- For example, the following code

```
try {  
    System.out.println(refVar.toString());  
}  
catch (NullPointerException ex) {  
    System.out.println("refVar is null");  
}
```

is better replaced by

```
if (refVar != null)  
    System.out.println(refVar.toString());  
else  
    System.out.println("refVar is null");
```

# Getting Information from Exception Object

- An exception object contains valuable information about the exception.
- You may use the following instance methods in the **java.lang.Throwable** class to get information regarding the exception:
  - ✓ **+getMessage(): String** - returns the descriptive string stored in an exception object.
  - ✓ **+printStackTrace(): void** - Prints the Throwable object and its call stack trace information on the console.
  - ✓ Helpful in testing and debugging.

## Cont'd...

- ✓ **+getStackTrace(): StackTraceElement[]** - Returns an array of stack trace elements representing the stack trace pertaining to this exception object.
  - *StackTraceElement*'s methods *getClassName*, *getFileName*, *getLineNumber* and *getMethodName* get the class name, file name, line number and method name, respectively, for that *StackTraceElement*.

# Example

```
1 public class GettingInfoDemo {
2     public static void main(String[] args) {
3         try{
4             throw new ClassCastException();
5         }
6         catch(ClassCastException e){
7             StackTraceElement [] err = e.getStackTrace();
8             System.out.println("Class\t\tFilename\t\tLineNumber\tMethod" );
9             for (StackTraceElement element : err)
10            {
11                System.out.printf("%s\t",element.getClassName() );
12                System.out.printf("%s\t", element.getFileName());
13                System.out.printf("%s\t", element.getLineNumber());
14                System.out.printf("%s%n", element.getMethodName());
15            }
16            System.out.println("Exception type: "+e.getClass());
17        }
18    }
19 }
```

| Class                                              | Filename             | LineNumber | Method |
|----------------------------------------------------|----------------------|------------|--------|
| GettingInfoDemo                                    | GettingInfoDemo.java | 4          | main   |
| Exception type: class java.lang.ClassCastException |                      |            |        |

# Defining Custom Exception Classes

- Sometimes it's useful to declare your own exception classes that are specific to the problems that can occur when another programmer uses your reusable classes.
- A new exception class must extend an existing **Exception** or a **subclass of Exception** class to ensure that the class can be used with the exception handling mechanism.

# Example: Custom Exception Class

```
1  import java.util.Scanner;
2  public class InvalidAgeException extends Exception {
3      public static void main(String[] args) {
4          System.out.println("Enter your age");
5          try{
6              // enter your age
7              Scanner sc = new Scanner(System.in);
8              double age = sc.nextDouble();
9              validateAge(age);
10
11          }
12          catch(InvalidAgeException ex){
13              System.out.println("Your age is not valid");
14          }
15      }
16      public static void validateAge(double age) throws InvalidAgeException {
17          if(age >= 18){
18              System.out.println("Your age is valid");
19          }
20          else{
21              throw new InvalidAgeException();
22          }
23      }
24  }
```

# Preconditions and Postconditions

- ❖ To improve the overall design, programmers can specify the expected states before and after a method's execution, called preconditions and postconditions respectively.
- ❖ A **precondition** must be true when a method is invoked just before it begins executing.
- ❖ A **postcondition** is true after the method successfully returns.
- ❖ As an example, examine String method **charAt**, which has one int parameter. For a precondition, method charAt assumes that index is greater than or equal to zero and less than the length of the String.
- ❖ If the precondition is met, the postcondition states that the method will return the character at the position in the String specified by the parameter index. Otherwise, the method throws an **IndexOutOfBoundsException**.

# Assertions

- ❖ An **assertion** is a Java statement that can be used to ensure that your code is working correctly.
- ❖ Preconditions and postconditions are two types of assertions.
- ❖ Assertions can be validated programmatically using the **assert** statement.
- ❖ You should not use `assert` to indicate runtime problems in production code—use the exception mechanism for this purpose.
- ❖ Java includes two versions of the *assert* statement for validating assertions programmatically:

# Assertions...

1. *assert* *boolean expression*;

☞ throws an **AssertionError** (a subclass of *Error*) if *boolean expression* is *false*.

2. *assert* *boolean expression1* : *expression2*;

☞ Evaluates *boolean expression1* and if false, it throws an **AssertionError** with *expression2* as the error message.

❖ By default assertions are disabled in JVM.

❖ You must explicitly enable assertions.

❖ To enable assertions, use the *java* command's **-ea** command line option, as in

***java -ea AssertTest***

Where AssertTest is the name of the class.

# Example

```
1 // Fig. 11.8: AssertTest.java
2 // Checking with assert that a value is within range
3 import java.util.Scanner;
4
5 public class AssertTest
6 {
7     public static void main(String[] args)
8     {
9         Scanner input = new Scanner(System.in);
10
11         System.out.print("Enter a number between 0 and 10: ");
12         int number = input.nextInt();
13
14         // assert that the value is >= 0 and <= 10
15         assert (number >= 0 && number <= 10) : "bad number: " + number;
16
17         System.out.printf("You entered %d\n", number);
18     }
19 } // end class AssertTest
```

Enter a number between 0 and 10: 5  
You entered 5

Enter a number between 0 and 10: 50  
Exception in thread "main" java.lang.AssertionError: bad number: 50  
at AssertTest.main(AssertTest.java:15)

Thank You